

Problem Set Solutions: Seminar 01

Asymptotics and Simple Algorithms
MIT-style solution format

Algorithms and Data Structures I

June 29, 2026

Course: Algorithms and Data Structures I
Topic: Asymptotics, prefix/difference arrays, two pointers, binary search
Style: Full problem-set solutions with algorithms, proofs, and complexity bounds

Why this matters. These problems are a compact toolkit for algorithm design. Difference arrays turn many interval updates into two endpoint updates; prefix sums turn range information into constants; two-pointers exploit monotonicity; binary search on the answer reduces optimization to feasibility; and merge-sort counting is the prototype for divide-and-conquer counting problems. The recurring theme is to replace repeated local work by a global invariant.

Contents

Problem 1. Truth or falsehood for asymptotic claims	3
Problem 2. Range additions in $O(n + q)$	3
Problem 3. The k -th zero starting from index i	4
Problem 4. Lexicographically minimum or maximum merge	4
Problem 5. Testing whether one sequence is a subsequence of another	5
Problem 6. Longest subarray with sum at most k	6
Problem 7. Existence of a subarray with exact sum s	6
Problem 8. Maximum number of points covered by one segment	7
Problem 9. Counting sorted values in a range	7
Problem 10. Range membership queries	8
Problem 11. Finding the peak of a bitonic array	8
Problem 12. Minimum banana-eating speed	9
Problem 13. Diplomas on a square wall	9
Problem 14. Maximizing the minimum after one sign-flip segment	10

Problem 15. Finding an unknown nonnegative integer by comparison queries **11**

Conventions. Arrays are indexed from 1 unless stated otherwise. Intervals $[l, r]$ are inclusive. Logs may have any fixed base greater than 1; changing the base changes only constant factors. For algorithms involving large products, use a sufficiently wide integer type.

Problem 1. Truth or falsehood for asymptotic claims

Decide whether each statement is true.

- (a) $2^{n+3} = \Theta(2^n)$.
- (b) $2^n = \Theta(2^{n/2})$.
- (c) $n^2 = O(2^n)$.
- (d) $n \log n = \Omega(\log n)$.
- (e) $n \log n = \Theta(n^2)$.

Answer.

Part	Truth value	Reason
(a)	True	$2^{n+3} = 8 \cdot 2^n$ differs by a constant factor.
(b)	False	$2^n / 2^{n/2} = 2^{n/2} \rightarrow \infty$.
(c)	True	Exponential growth eventually dominates every fixed polynomial.
(d)	True	For $n \geq 2$, $n \log n \geq \log n$.
(e)	False	$(n \log n) / n^2 = (\log n) / n \rightarrow 0$.

Problem 2. Range additions in $O(n + q)$

Initially there is an array $a_1, \dots, a_n$. There are q transformations (l, r, x) , meaning that all elements $a_l, \dots, a_r$ must be increased by x . Output the final array in $O(n + q)$ time.

Solution.

Idea. A range update is expensive in the original array but cheap in the neighboring-difference array. Define

$$d_1 = a_1, \quad d_i = a_i - a_{i-1} \quad (2 \leq i \leq n).$$

Then $a_i = d_1 + d_2 + \dots + d_i$.

Algorithm. Construct d . For each update (l, r, x) do

$$d_l \leftarrow d_l + x,$$

and, if $r < n$, do

$$d_{r+1} \leftarrow d_{r+1} - x.$$

After all updates, recover the final array by prefix sums:

$$a'_i = d_1 + d_2 + \dots + d_i.$$

Correctness. Consider one update (l, r, x) . Adding x to d_l increases every reconstructed prefix $a'_i = \sum_{t=1}^i d_t$ for $i \geq l$ by x . If $r < n$, subtracting x from d_{r+1} cancels this increase for every $i \geq r + 1$. Hence exactly the reconstructed entries with indices $l \leq i \leq r$ increase by x , and all other entries are unchanged. By linearity, applying this endpoint rule for all updates gives exactly the same result as applying the q range additions directly. The final prefix-sum reconstruction therefore outputs the desired final array.

Running time. Constructing d takes $O(n)$ time. Each update takes $O(1)$ time. Reconstruction takes $O(n)$ time. Total time is $O(n + q)$ and memory is $O(n)$.

Problem 3. The k -th zero starting from index i

Given an array $a_1, \dots, a_n$ consisting only of zeros and ones, answer q queries: find the position of the k -th zero starting from index i . Process all queries in $O(n + q)$ time.

Solution.

Algorithm. Preprocess two arrays:

- $pref[t]$ = number of zeros among $a_1, \dots, a_t$;
- $Z[1], Z[2], \dots, Z[z]$ = positions of all zeros in increasing order.

For a query (i, k) , the number of zeros strictly before i is $pref[i - 1]$. Therefore the requested zero is the

$$target = pref[i - 1] + k$$

-th zero in the whole array. If $target > z$, return “does not exist”; otherwise return $Z[target]$.

Correctness. The list Z stores all zero positions in increasing order, so its t -th element is exactly the position of the t -th zero in the whole array. A zero is counted in the query only if its position is at least i . Since exactly $pref[i - 1]$ zeros lie before i , the k -th zero starting from i is the $(pref[i - 1] + k)$ -th zero globally. The algorithm returns precisely that position, or correctly reports nonexistence when that global zero number exceeds the total number of zeros.

Running time. The preprocessing scan is $O(n)$. Each query uses constant-time array accesses, so q queries take $O(q)$. Total time is $O(n + q)$ and memory is $O(n)$.

Problem 4. Lexicographically minimum or maximum merge

Let $a_1, \dots, a_n$ and $b_1, \dots, b_m$ be two sequences. Merge them into one sequence while preserving the relative order inside each input sequence. Produce the lexicographically minimum or maximum possible merged sequence in $O(n + m)$ time. Assume all numbers are pairwise distinct.

Solution.

Algorithm. Maintain pointers i and j to the first unused elements of a and b .

- For the lexicographically minimum merge: at each step output the smaller of the two available heads a_i and b_j .
- For the lexicographically maximum merge: at each step output the larger of the two available heads a_i and b_j .

If one sequence is exhausted, append the rest of the other sequence.

Correctness. At any step before one sequence is exhausted, the next element of any valid merge must be either the current head a_i or the current head b_j ; no later element can be output before its predecessors. For the minimum merge, suppose $a_i < b_j$. Every valid merge beginning with b_j is lexicographically larger than every valid merge beginning with a_i , because the first position already differs and $a_i < b_j$. Thus any lexicographically minimum merge must choose a_i . The same argument applies symmetrically when $b_j < a_i$. Repeating this forced choice proves that the greedy algorithm constructs the lexicographically minimum merge. For the maximum merge the comparison is reversed: choosing the larger available head is forced by the same first-difference argument. Once one input is exhausted, the remaining elements have only one possible relative order, so appending them is necessary.

Running time. Each element is output exactly once, and each pointer only moves forward. The running time is $O(n + m)$ and the extra memory, excluding the output, is $O(1)$.

Problem 5. Testing whether one sequence is a subsequence of another

Let $a = (a_1, \dots, a_n)$ and $b = (b_1, \dots, b_m)$. Determine in $O(m)$ time whether a is a subsequence of b .

Solution.

Algorithm. Use a pointer j to the first not-yet-matched element of a , initially $j = 1$. Scan b from left to right. Whenever $b_i = a_j$, match this element and increment j . If $j = n + 1$, all elements of a have been matched, so return true. If the scan ends before this happens, return false.

Correctness. The greedy rule matches each required element a_j to the earliest possible position in b after the previous match. This can never hurt: if some valid subsequence embedding matches a_j at a later position, replacing that position by the earlier greedy match preserves the order of all future possible matches and leaves at least as much suffix of b available. By induction over j , after each greedy match there exists a valid embedding of the matched prefix whenever any embedding exists at all. Therefore, if the greedy scan matches all of a , then a is a subsequence of b ; if it fails, no valid embedding exists.

Running time. The scan over b is linear and the pointer j increases at most n times. The running time is $O(m)$ and the extra memory is $O(1)$.

Problem 6. Longest subarray with sum at most k

Given positive numbers $a_1, \dots, a_n$, determine in $O(n)$ time the length of the longest contiguous subarray whose sum is at most k .

Solution.

Algorithm. Use a sliding window $[l, r]$ and its current sum S . Initialize $l = 1$, $r = 0$, $S = 0$, and $ans = 0$. For $l = 1, 2, \dots, n$:

1. While $r < n$ and $S + a_{r+1} \leq k$, set $r \leftarrow r + 1$ and $S \leftarrow S + a_r$.
2. Update $ans \leftarrow \max(ans, r - l + 1)$.
3. Before increasing l , if $r \geq l$, set $S \leftarrow S - a_l$; otherwise set $r \leftarrow l$ and keep $S = 0$.

Return ans .

Correctness. Because all a_i are positive, increasing r strictly increases the window sum. Thus, for a fixed l , once $S + a_{r+1} > k$, no larger right endpoint is feasible. Step 1 therefore finds the maximum feasible r for the current l . When l increases to $l + 1$, removing a_l only decreases the sum; hence the maximum feasible right endpoint for the new left boundary cannot lie to the left of the old one. Therefore the pointer r only needs to move forward. Since the algorithm records the best feasible window for every possible left boundary, it returns the maximum length over all feasible subarrays.

Running time. Both l and r move from left to right at most n times. The total time is $O(n)$ and the extra memory is $O(1)$.

Problem 7. Existence of a subarray with exact sum s

Given nonnegative integers $a_1, \dots, a_n$ and a number s , determine whether some nonempty contiguous subarray has sum exactly s .

Solution.

Algorithm. Use a sliding window. Set $l = 1$ and $S = 0$. For $r = 1, 2, \dots, n$:

1. Add the new right endpoint: $S \leftarrow S + a_r$.
2. While $l \leq r$ and $S > s$, remove the left endpoint: $S \leftarrow S - a_l$, $l \leftarrow l + 1$.
3. If $l \leq r$ and $S = s$, return true.

If the loop finishes, return false.

Correctness. For a fixed right endpoint r , after the while-loop terminates, the current window has sum at most s . Any earlier left endpoint that has already been discarded produced a sum greater than s at the moment it was discarded; since all future additions are nonnegative, it can never later form a sum exactly s with a larger right endpoint before being discarded. Thus no candidate subarray is lost. Whenever the current sum equals s , the current nonempty window is a valid subarray. Conversely, if a valid subarray exists, its left endpoint is not discarded before its right endpoint is processed, because its sum is exactly s and not greater than s ; when that right endpoint is reached, the algorithm detects a window of sum s .

Running time. Each endpoint moves right at most n times. The running time is $O(n)$ and the extra memory is $O(1)$.

Problem 8. Maximum number of points covered by one segment

On the line there are sorted points $x_1 \leq x_2 \leq \dots \leq x_n$ and a number y . Determine the maximum number of points that can be covered by one segment of length y .

Solution.

Algorithm. Use two pointers. For each left endpoint index $l = 1, 2, \dots, n$, maintain the largest r such that

$$x_r - x_l \leq y.$$

Increase r while $r < n$ and $x_{r+1} - x_l \leq y$. Then update

$$ans \leftarrow \max(ans, r - l + 1).$$

Correctness. For any segment covering points from l through r , its length must be at least $x_r - x_l$. Therefore, for a fixed leftmost covered point x_l , the best possible rightmost point is the largest r satisfying $x_r - x_l \leq y$. The algorithm computes exactly this r for every l . When l increases, the condition $x_r - x_l \leq y$ becomes no harder for the old r , so the optimal r never moves left. Thus two pointers enumerate all optimal windows, and the maximum recorded window size is the answer.

Running time. Both pointers move at most n times, so the algorithm runs in $O(n)$ time and uses $O(1)$ extra memory.

Problem 9. Counting sorted values in a range

Given a sorted array $a_1 \leq a_2 \leq \dots \leq a_n$, answer queries: how many array values lie in the range $[x, y]$?

Solution.

Algorithm. For each query, compute

$$L = \text{lower_bound}(a, x),$$

the first index with $a_L \geq x$, and

$$R = \text{upper_bound}(a, y),$$

the first index with $a_R > y$. Then the number of elements in $[x, y]$ is

$$R - L.$$

If using one-based indexing, interpret these bounds as positions in the half-open index range $[1, n + 1)$.

Correctness. All elements before L are strictly smaller than x , so none of them should be counted. All elements from L up to $R - 1$ satisfy $x \leq a_i \leq y$, so all of them should be counted. All elements from R onward are strictly greater than y , so none of them should be counted. Hence exactly $R - L$ elements lie in the query range.

Running time. Each binary search takes $O(\log n)$ time, so each query takes $O(\log n)$ time. The extra memory is $O(1)$.

Problem 10. Range membership queries

Given an array $a_1, \dots, a_n$ of integers from 1 to n , answer queries of the form: is there an index $t \in [l, r]$ such that $a_t = x$?

Solution.

Algorithm. Preprocess occurrence lists. For every value $x \in \{1, \dots, n\}$, create a sorted list

$$Pos[x] = \{t : a_t = x\}$$

by scanning the array once and appending each index to the list of its value.

To answer query (l, r, x) , binary search in $Pos[x]$ for the first position $p \geq l$. If such a p exists and $p \leq r$, answer yes; otherwise answer no.

Correctness. The list $Pos[x]$ contains exactly the indices where x appears, in increasing order. If the first occurrence of x at or after l is at position $p \leq r$, then $p \in [l, r]$ and the answer is yes. If no such p exists, or if the first such p is greater than r , then every occurrence of x is either before l or after r , so no index in $[l, r]$ contains x . Therefore the query answer is correct.

Running time. The preprocessing scan takes $O(n)$ time and $O(n)$ memory in total. Each query takes $O(\log |Pos[x]|) \leq O(\log n)$ time.

Problem 11. Finding the peak of a bitonic array

The array $a_1, \dots, a_n$ has the property that for some index i ,

$$a_1 < a_2 < \dots < a_i > a_{i+1} > \dots > a_n.$$

Find such an index i in $O(\log n)$ time.

Solution.

Algorithm. Binary search on the slope. Set $lo = 1$, $hi = n$. While $lo < hi$:

1. Set $mid = \lfloor (lo + hi)/2 \rfloor$.
2. If $a_{mid} < a_{mid+1}$, then the peak is to the right, so set $lo = mid + 1$.
3. Otherwise $a_{mid} > a_{mid+1}$, so the peak is at mid or to the left; set $hi = mid$.

Return lo .

Correctness. If $a_{mid} < a_{mid+1}$, the array is still increasing at mid , so the peak cannot be at or left of mid ; it must lie in $[mid + 1, hi]$. If $a_{mid} > a_{mid+1}$, the array is decreasing immediately after mid , so the peak cannot lie strictly to the right of mid ; it lies in $[lo, mid]$. Thus each iteration preserves the invariant that the true peak index belongs to $[lo, hi]$. The interval length strictly decreases, so the loop terminates with $lo = hi$, and by the invariant this index is the peak.

Running time. Each iteration halves the search interval. The running time is $O(\log n)$ and the extra memory is $O(1)$.

Problem 12. Minimum banana-eating speed

There are n piles of bananas; pile i contains a_i bananas. The monkey chooses an integer speed S and eats up to S bananas per minute from a single pile. If a pile has fewer than S bananas left, the monkey eats the rest of that pile in that minute. The owner returns after M minutes. Find the minimum S such that all bananas can be eaten within M minutes.

Solution.

Idea. For a fixed speed S , pile i takes

$$\left\lceil \frac{a_i}{S} \right\rceil$$

minutes. Hence the total time is

$$T(S) = \sum_{i=1}^n \left\lceil \frac{a_i}{S} \right\rceil.$$

The function $T(S)$ is monotone nonincreasing in S .

Algorithm. Binary search the smallest feasible S in the interval $[1, \max_i a_i]$. A candidate speed S is feasible exactly when

$$\sum_{i=1}^n \left\lceil \frac{a_i}{S} \right\rceil \leq M.$$

Compute each ceiling as $(a_i + S - 1) // S$ in integer arithmetic.

Correctness. If speed S is feasible, then any larger speed $S' > S$ is also feasible because no pile can take more minutes at a larger speed. Therefore the feasible speeds form a suffix of the integer interval $[1, \max_i a_i]$. Binary search on a monotone predicate returns the first element of this suffix, which is exactly the minimum speed allowing the monkey to finish within M minutes.

Running time. One feasibility check costs $O(n)$ time. The search interval has size at most $\max_i a_i$, so the total time is $O(n \log \max_i a_i)$ and the extra memory is $O(1)$.

Problem 13. Diplomas on a square wall

Petya has n diplomas, each a rectangle of size $w \times h$. They cannot be rotated or overlap. Determine the minimum side length a of a square wall $a \times a$ on which all diplomas fit.

Solution.

Idea. For a fixed square side a , the number of diplomas that fit in a grid is

$$\left\lfloor \frac{a}{w} \right\rfloor \cdot \left\lfloor \frac{a}{h} \right\rfloor.$$

This quantity is monotone nondecreasing in a .

Algorithm. Binary search the smallest a satisfying

$$\left\lfloor \frac{a}{w} \right\rfloor \cdot \left\lfloor \frac{a}{h} \right\rfloor \geq n.$$

A simple initial upper bound is $hi = n \max(w, h)$, because then one can place all diplomas in one column or one row. Set $lo = 0$ and binary search on the half-open invariant: lo is infeasible, hi is feasible.

Correctness. For any fixed a , the grid with $\lfloor a/w \rfloor$ columns and $\lfloor a/h \rfloor$ rows is feasible and contains exactly their product many diplomas. Thus the predicate above is sufficient. It is also necessary for this axis-aligned nonrotated grid packing criterion used by the problem: no more than $\lfloor a/w \rfloor$ diplomas fit horizontally in any row and no more than $\lfloor a/h \rfloor$ rows fit vertically. Since the predicate is monotone in a , binary search returns its smallest feasible value, which is the desired minimum square side.

Running time. Each feasibility check is $O(1)$. With upper bound $n \max(w, h)$, binary search uses $O(\log(n \max(w, h)))$ checks and $O(1)$ memory.

Problem 14. Maximizing the minimum after one sign-flip segment

Given an array $a_1, \dots, a_n$ with at least one negative number, one operation chooses a nonempty contiguous segment and replaces every number in it by its opposite. Find the maximum possible value of $\min_i a_i$ after one operation.

Solution.

Idea. Binary search the answer. For a candidate value x , ask whether it is possible to make every final element at least x after flipping one segment.

If an element is outside the flipped segment, it remains a_i and must satisfy $a_i \geq x$. Therefore every index with $a_i < x$ must be inside the flipped segment. If an element is inside the flipped segment, it becomes $-a_i$ and must satisfy $-a_i \geq x$, equivalently $a_i \leq -x$. Therefore no index with $a_i > -x$ may be inside the flipped segment.

Algorithm. Let

$$B_x = \{i : a_i < x\}$$

be the set of indices that must be flipped.

- If $B_x = \emptyset$, then x is feasible: all elements already satisfy $a_i \geq x$, and since the original array contains a negative number, flipping one sufficiently negative single element preserves the lower bound x .

- Otherwise let $L = \min B_x$ and $R = \max B_x$. Any feasible segment must contain every index in B_x , so it must contain $[L, R]$. The candidate x is feasible iff every index $t \in [L, R]$ satisfies $a_t \leq -x$.

Then binary search the maximum feasible integer x over the range

$$[-A, A], \quad A = \max_i |a_i|.$$

Correctness. First fix a candidate x . Any index with $a_i < x$ would be too small if left outside the flipped segment, so all such indices must be included. Hence every feasible segment must contain the interval from the leftmost such index L to the rightmost such index R . Conversely, if every element in $[L, R]$ satisfies $a_t \leq -x$, then flipping exactly the segment $[L, R]$ makes all elements inside it at least x , while every element outside it was not in B_x and therefore already satisfies $a_i \geq x$. Thus the feasibility test is exact. If B_x is empty, all elements already satisfy the lower bound; because at least one original element is negative, flipping a single negative element gives a positive value and does not violate any lower bound $x \leq \min_i a_i$.

The predicate “ x is feasible” is monotone: if all final elements can be made at least x , then the same operation also makes all final elements at least any $y \leq x$. Therefore binary search over integer x returns the maximum feasible lower bound, which is exactly the largest possible value of the final minimum.

Running time. A feasibility check scans the array once, so it costs $O(n)$ time. The search interval has length $2A + 1$, so the total time is $O(n \log A)$ and the extra memory is $O(1)$.

Problem 15. Finding an unknown nonnegative integer by comparison queries

An unknown integer $n \geq 0$ is fixed. You may ask questions of the form: “is $n \leq a$?” for any integer a . Determine n using $O(\log n)$ queries.

Solution.

Algorithm. First find a power-of-two upper bound by exponential search.

1. Set $hi = 1$.
2. While the answer to “is $n \leq hi$?” is no, set $hi \leftarrow 2hi$.

Now $n \leq hi$. If $hi > 1$, the previous failed query implies $n > hi/2$. Thus $n \in [hi/2 + 1, hi]$. If $hi = 1$, then $n \in [0, 1]$. Binary search this final interval using the same predicate “ $n \leq a$ ” until one value remains.

Correctness. The exponential phase terminates because powers of two grow without bound. At termination, hi is an upper bound for n . If there was a previous value $hi/2$, the query at $hi/2$ failed, so $n > hi/2$; hence the maintained interval contains n . The predicate $P(a) = [n \leq a]$ is monotone: once it becomes true, it remains true for all larger a . Binary search finds the smallest a for which $P(a)$ is true, and that smallest value is exactly n .

Running time. If $n \geq 2$, the exponential phase asks $O(\log n)$ queries before finding $hi < 2n$. The binary search interval then has size $O(n)$ and also takes $O(\log n)$ queries. For $n = 0$ or $n = 1$, the algorithm uses $O(1)$ queries. Thus the total number of queries is $O(\log n)$, with the standard convention that the small cases are constant.